

Modül 07: Arayüzler

Bir sınıf tam olarak bir sınıfı genişletebilir ve istediği kadar arayüz uygulayabilir. Bu modüldeki her şey o asimetriden çıkar.

Modül, bu müfredattaki kalıtıma karşı en güçlü argümanla bitiyor, ve bu bir slogan değil. Standart kütüphaneden iki sınıfın, birebir aynı koda farklı cevap vermesi.

Neler Öğreneceksiniz

- Yetenek olarak arayüzler ve neden işi gören en dar olanı kabul etmeniz gerektiği
- Bir arayüzün soyut sınıfı ne zaman yendiği, ne zaman yenmediği
- Varsayılan metotlar ve icat edilme sebepleri olan o belirli sorun
- Statik ve özel arayüz metotları
- Elmas probleminin neden tahmin değil derleme hatası olduğu
- "Kompozisyonu tercih edin" sözünün sizi gerçekte neden koruduğu

Başka Bir Dilden Geliyorsanız

	Python	Java
Sözleşme	ördek tiplere ya da <code>Protocol</code>	derleme zamanında denetlenen <code>interface</code>
Çoklu ebeveyn	tam çoklu kalıtım	bir sınıf, çok arayüz
Mixin	çoklu kalıtımla	varsayılan metotlu arayüzlerle
Çakışma çözümü	MRO sessizce karar verir	derleme hatası, siz karar verirsiniz

Python belirsizliği metot çözümleme sırasıyla sizin için çözer. Java çözmeyi reddeder, çünkü iki eşit derecede makul davranış arasında sessiz bir seçim, kimsenin açıklayamayacağı hataların çıkış yoludur.

Ders

Arayüzler yetenektir

`Interfaces.java` dosyasından:

```
private static void launch(Flies thing) {  
    System.out.println("launching: " + thing.fly());  
}
```

O metot bir ördek, bir uçak ve sonradan birinin yazacağı her şey üzerinde çalışır. Nesnenin ne *olduğu* umurunda değil, yalnızca uçabildiği.

```
launching: duck flapping
launching: jet engines
```

Swims ve Speaks uygulayan ama Flies uygulamayan bir Penguin geçirilirse derlenmez. Yetenek varsayılmaz, denetlenir.

İşi gören en dar yeteneği kabul edin. Flies tipli bir parametre henüz var olmayan tipleri alabilir. Duck tipli olan alamaz.

Arayüz mü soyut sınıf mı?

	arayüz	soyut sınıf
Sınıf başına kaç tane	istediğiniz kadar	tam olarak bir
Örnek alanları	yok	var
Kurucular	yok	var
Durum	yok	var

Neyi modellediğinizi sorun. **"Yapabilir" bir arayüzdür:** Flies , Comparable , AutoCloseable . **"Bir çeşididir" bir soyut sınıftır**, ve yalnızca alt sınıflar gerçekten durum ve kurulum mantığı paylaşıyorsa.

İkisi de uyuyorsa arayüzü tercih edin. Uygulayıcının tek kalıtım yuvasını boş bırakır, ve o yuva kolayca harcanıp sonradan pişman olunan bir şeydir.

Varsayılan metotlar ve var olma sebepleri

DefaultMethods.java dosyasından:

```
interface Collection {
    List<String> tracks(); // must implement

    default int count() { return tracks().size(); } // free
    default boolean isEmpty() { return count() == 0; }
}
```

Buradaki tarih önemli. Java 8'den önce bir arayüze metot eklemek, her yerdeki mevcut her uygulamayı anında kırıyordu. Genel kütüphanelerdeki arayüzler fiilen sonsuza dek donmuştu.

Java 8'in java.util.Collection arayüzüne stream() eklemesi gerekiyordu. Varsayılan metotlar olmasaydı o tek ekleme, herhangi biri tarafından yazılmış her koleksiyon sınıfını kırardı.

Varsayılan metotlar kütüphane evrimi için vardır. Çoklu kalıtımı gizlice içeri sokmanın yolu değil, ve öyle kullanmak bir sonraki bölümü üretir.

İki küçük ekleme:

- **Özel arayüz metotları** (Java 9) birkaç varsayılanın, onu uygulayan herkese göstermeden yardımcı kod paylaşmasını sağlar.
- **Statik arayüz metotları** arayüzün kendisine aittir. Kütüphanede `List.of`, `Map.of` ve `Comparator.comparing` bulunmasının sebebi budur.

Elmas problemi

İki arayüz, aynı varsayılan, ikisini de uygulayan bir sınıf. `DiamondConflict.java`:

```
error: types Timestamped and Versioned are incompatible;
class Entry implements Timestamped, Versioned {
^
  class Entry inherits unrelated defaults for describe() from types Timestamped and
  Versioned
```

Java tam olarak bundan kaçınmak için yirmi yıl çoklu sınıf kalıtımını yasakladı. Varsayılan metotlar dar bir sürümünü geri getirdi, dolayısıyla dil bunu tek güvenli yolla ele alıyor: karar vermenizi sağlıyor.

```
@Override
public String describe() {
    return Timestamped.super.describe() + " / " + Versioned.super.describe();
}
```

`InterfaceName.super.method()` sözdizimi yalnızca bunun için var. Yalnızca bir arayüz varsayılan sağladığında ya da sınıf kendi uygulamasını verdiğinde çakışma olmaz.

Kompozisyonun sizi gerçekte neden koruduğu

Hatırlanmaya değer kısım bu. `CompositionOverInheritance.java` sayan bir sınıfı üç kez yazıyor. Gerçek çıktı:

```
--- inheritance, extending HashSet ---
added 3, counter says: 6    <- wrong

--- the SAME code, extending ArrayList ---
added 3, counter says: 3    <- right, by luck

--- composition ---
added 3, counter says: 3    <- right, by design
```

Her iki alt sınıftaki geçersiz kılma birebir aynı:

```
@Override public boolean add(String item)    { added++; return super.add(item); }
@Override public boolean addAll(Collection<? extends String> items) {
    added += items.size();
    return super.addAll(items);
}
```

`HashSet.addAll` her eleman için `this.add` çağırır, dolayısıyla geçersiz kılma tekrar çalışır ve sayı ikiye katlanır. `ArrayList.addAll` toplu kopyalar, dolayısıyla katlanmaz.

Aynı kütüphaneden iki sınıf, aynı kod, farklı cevaplar. Ve alt sınıf kendi kaynağına bakarak nedenini göremez.

İşte bu **kırılğan temel sınıf** problemi. Sınıfınızın doğruluğu, üst sınıfın hiç söz vermediği ve herhangi bir sürümde değiştirebileceği bir uygulama ayrıntısına bağlı. Kompozisyon bağımlılığı tamamen kaldırır: koleksiyonu tutun, koleksiyon olmayın.

Bedeli, kırk metodu miras almak yerine dışarı açmak istediklerinizi yazmanızdır. O bedel genellikle ödemeye değer, ve iyi bir soru sordurur: bu tipin o kırk metottan hangisini sunması gerekiyordu?

Çalıştırın

```
java modules/07-interfaces/examples/Interfaces.java
java modules/07-interfaces/examples/DefaultMethods.java
java modules/07-interfaces/examples/CompositionOverInheritance.java

# Bilerek başarısız olur.
java modules/07-interfaces/examples/DiamondConflict.java

./scripts/verify-examples.sh modules/07-interfaces
```

Sık Yapılan Hatalar

Arayüz yeterken parametreyi bir sınıfa bağlamak. Sizden sonra yazılan her tipi dışarıda bırakır.

Varsayılan metotlarla mixin kurmak. Kütüphane evrimi için yapıldılar. Arayüzler hâlâ durum tutamaz, dolayısıyla hatırlanacak bir şeyi olan "mixin" zaten çalışmaz.

Davranış eklemek için bir kütüphane sınıfını genişletmek. Sayaç örneği ne olduğunu gösteriyor. Onun yerine sarın.

Arayüz metotlarının örtük olarak public olduğunu unutmak. Birini daha dar görünürlikle uygulamak derleme hatasıdır ve mesaj ilk seferinde apaçık değildir.

"İleride paylaşılan kod olabilir" diye soyut sınıfa uzanmak. Arayüzle başlayın. Altına sonradan soyut sınıf eklemek kolaydır; uygulayıcının kalıtım yuvasını geri almak değildir.

Özet Çıkarımlar

- **Bir üst sınıf, çok arayüz.** Arayüzlerin var olma sebebi bu sınırdır.
- **İşi gören en dar yeteneği kabul edin**, böylece gelecekteki tipler onu karşılayabilsin.
- **"Yapabilir" arayüz, "bir çeşididir" sınıftır.** İkisi de uyuyorsa arayüzü alın.
- **Varsayılan metotlar kütüphanelerin uygulayıcıları kırmadan büyümesi için var.**
- **Çakışan varsayılanlar derleme hatasıdır**, `Interface.super.method()` ile çözülür.
- **Kalıtım sizi üst sınıfın API'sine değil uygulamasına bağlar.** Aynı geçersiz kılma `ArrayList` altında doğru, `HashSet` altında yanlış.

Ödev

[homework/README.md](#)

Yeteneklerle tasarlayın, sonra kırılğan temel sınıf hatasını yeniden üretip kompozisyonla düzeltin. Referans çözüm [solutions/07-interfaces/](#) altında.